

Lesson 8: More SQL

Getting Unique Values

- There are many use cases for being able to query the unique values from a specific column.
- We use the DISTINCT clause give us a results set of only the unique value of a specific column

```
/*Return the unqiue values from the id_column column*/  
SELECT DISTINCT id_column  
FROM database.table
```

Aggregate Functions

- These functions are used in with the `SELECT` command to provide a mathematical summary of the data
- A column name is the only argument that is passed to these functions
- Make sure that you read the documentation to understand how NULL values are treated by these functions

Aggregate Functions continued

Function	Definition	General Data Types
COUNT()	The number of rows in a results set	Any
MIN()	The minimum value in a given column	Numeric
MAX()	The maximum value in a given column	Numeric
SUM()	The sum of all values in a given column	Numeric
AVG()	The average or mean of all values in a given column	Numeric

ORDER BY Clause

- We've discussed the `SELECT` command and the need to have a `FROM` clause and the option to have a `WHERE` clause
- There are some more optional clauses that you can have when you're querying data
- One of these is the `ORDER BY`, which as it sounds orders your results set

ORDER BY Clause continued

- Order by can sort data in `ASCENDING` (small to large) or `DESCENDING` (large to small)
 - The default order depends on the DBMS, but is generally ascending
 - These keywords can also be shortened to `ASC` and `DESC`
- Keep in mind that the placement of NULL values is determined by the DBMS

GROUP BY Clause

- There are lots of times where we want to summarize our data by different groups, rather than generally (aggregate function only)
- For these cases, we use the `GROUP BY` clause to define the column(s) that we want to group our data by
- The other columns that are defined in the `SELECT` that are not in the `GROUP BY` must be defined with an aggregate function.

```
SELECT id_column  
       ,SUM(value_column)  
FROM database.table  
GROUP BY id_column
```

HAVING Clause

- This is an optional clause that is used in conjunction with a `GROUP BY` clause
 - It cannot be used without a `GROUP BY` clause!
- Think of it as a way of filtering your grouped results or a `WHERE` clause for your `GROUP BY`

```
SELECT id_column
       ,SUM(value_column)
FROM database.table
GROUP BY id_column
HAVING SUM(value_column) > 10
```


Query Definition vs. Execution Order

Command/Clause	Query Order	Execution Order
SELECT	1	5
FROM	2	1
WHERE	3	2
GROUP BY	4	3
HAVING	5	4
ORDER BY	6	6

Joining Tables

- In data modeling and the creation of our ERD and subsequently our database, we've done a lot of work ensuring referential integrity between our entities
- Normalized data doesn't always make sense to end users, so we need to denormalize that data
 - We typically do this by joining different entities together
- **Please ignore section 7-7 (pg 300-304) in the book, we shouldn't join tables this way!**
- **Please also ignore section 8-1c (pg 344), we won't do joins with USING!**

Joining Tables Continued

Join Type	Join Description
OUTER JOIN	Retains matching rows and unmatched rows (dependent on definition)
INNER JOIN	Only retain matching rows of defined criterion
NATURAL JOIN	
CROSS JOIN	Create a cartesian product between two tables

Joins in Practice

- In practice, INNER JOINS and OUTER JOINS will be used the vast majority of the time
- There are some analytical circumstances where the CROSS JOIN is useful
- I have never used a NATURAL JOIN

UNION and UNION ALL

- Recall the discussion of the `UNION` when in relational algebra
 - `UNION` stacks or concatenates data from each table and deduplicates the result set
 - `UNION ALL` does the same thing, but doesn't deduplicate the result set
- The tables need to be union compatible or have equivalent columns and data types

```
SELECT column1  
       ,column2  
FROM database.table  
UNION  
SELECT column1  
       ,column2  
FROM database.table2;
```

INTERSECT

- We also discussed the use of `INTERSECT` in relational algebra
 - `INTERSECT` returns only the rows in common between tables and deduplicates the result set
- The tables need to be union compatible or have equivalent columns and data types

```
SELECT column1
       ,column2
FROM database.table
INTERSECT
SELECT column1
       ,column2
FROM database.table2;
```

EXCEPT

- We also discussed the use of `EXCEPT` in relational algebra
 - `EXCEPT` returns only the rows that are not in common between tables and deduplicates the result set
- The tables need to be union compatible or have equivalent columns and data types

```
SELECT column1
       ,column2
FROM database.table
EXCEPT
SELECT column1
       ,column2
FROM database.table2;
```

Subqueries

- Subqueries are used in the creation of more complex queries
 - These queries involve the use of result sets in subsequent queries
- Sometimes you need subqueries to enforce the proper order of operations
- Note that **WHERE** , **IN** and **HAVING** subqueries could and should generally be replaced with joins

Subqueries Continued

- Subqueries are made up of the a few parts
 - The Outer Query (not in parentheses)
 - The Inner Query (generally in parentheses)
 - The Inner Query is executed first (let's look at the execution order)

Correlated Subqueries

- A specific type of subquery that executes once per row of data
 - Similar to a nested loop in code
- Again not necessarily computationally efficient

Formatting Best Practices Continued

- Use leading commas in `GROUP BY` when using multiple columns
- Use column aliases for calculated columns
- Use table aliases when joining tables and using the `AS` keyword
- Use table aliases when using subqueries (when applicable) and using the `AS` keyword

Homework

- Study for the midterm which is next Monday 3/10
- While this test will not be focused on the book, you should be caught up with reading
- The main focus will be:
 - Specific things that I dove deeper into within lectures or reiterated multiple times
 - What we've did in lab